

Mobile Workflow API Request — Login Token + Tenant + Type

For the Workflow Development Team

Mobile needs to call Workflow APIs using the **login access token** that is already available after authentication and retrieve the correct workflow based on **tenant ID + feature type**.

This approach should **not require the user to Join a Tenant just to access the Workflow APIs**. Since mobile already has the login access token, we should be able to pass that token directly to the Workflow API along with the required `tenant_id` and `type`.

This is also **not related to Join Tenant session expiry**.

1. What We Want

Authentication

Mobile already receives a **login access token** after successful authentication.

We would like the Workflow APIs to accept this existing login access token directly.

Do we need to join a tenant just to access the Workflow APIs?

Instead of:

1. Joining a tenant.
2. Getting a separate access token from the Join Tenant API.
3. Using that separate token to access the Workflow APIs.

Can we directly use the **existing login access token** and pass the required `tenant_id` and `type` in the Workflow API request?

This would simplify the mobile implementation because the login access token is already available.

If a workflow is tenant-specific, mobile will pass the corresponding `tenant_id` in the request. If the workflow is not tenant-specific, the login access token alone should be sufficient.

Which workflow should be returned?

Returning only one latest published workflow under a tenant is not sufficient.

Different features may require completely different workflows, for example:

- Product purchase
- Service booking

- Event registration
- Course enrolment
- Training
- Other future business processes

Therefore, when creating a workflow on the web, we should save a **type** for the workflow.

Mobile will then send:

`tenant_id + type`

The API should return the **latest published workflow for that specific tenant and type**.

2. Main API We Need

Endpoint

GET `/api/v1/workflows`

Request

Header:

Authorization: Bearer <login access token>

Query parameters:

- `tenant_id`
- `type`

Expected Behaviour

Item	How it should work
Authentication	Accept the existing login access token
<code>tenant_id</code>	Query parameter identifying the tenant associated with the selected product/
<code>type</code>	Query parameter identifying the workflow/feature type
Result	Return the latest published workflow for that <code>tenant_id + type</code>
Workflow content	Include the workflow steps, fields, conditions, rules, actions, instructions, and other required configuration
Version	Not required for mobile. Always return the latest published version
No workflow found	Return empty data. Mobile will fall back to its existing default/static flow

Example — Buy Now on a Product

GET /api/v1/workflows?tenant_id={tenantId}&type=product

Authorization: Bearer <login access token>

The response should contain only the **latest published product workflow for that tenant**.

It should not return an event, service, course, or any other workflow belonging to the same tenant.

3. Type Values

The **type** should be configured when creating the workflow on the web.

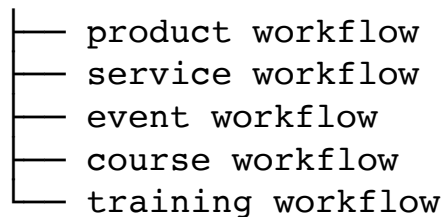
Type	Meaning	Mobile Example
product	Product purchase / Buy Now checkout	User taps Buy Now on a product from that tenant
service	Service booking flow	User books a service
enterprise	Enterprise flow	User opens an enterprise process
event	Event registration / booking flow	User registers for an event
course	Course enrolment flow	User enrolls in a course
training	Training flow	User starts a training process

Rule

Under one tenant, each **type** should have its own workflow.

For example:

Tenant A



Mobile should always receive:

Latest Published Workflow + matching `tenant_id` + matching `type`

Therefore, a **product** workflow and an **event** workflow should not share the same workflow/form configuration.

4. How Mobile Will Call It

User Action	What Mobile Sends
Buy Now on a product	Login access token + <code>tenant_id</code> of that product + <code>type=product</code>
Book a service	Login access token + <code>tenant_id</code> of that service + <code>type=service</code>
Register for an event	Login access token + <code>tenant_id</code> of that event + <code>type=event</code>
Enrol in a course	Login access token + <code>tenant_id</code> of that course + <code>type=course</code>
Start a training	Login access token + <code>tenant_id</code> of that training + <code>type=training</code>

The mobile application should not need to call **Join Tenant** simply to obtain another token for accessing these Workflow APIs.

5. Pre-Login — Login and Registration Fields

We also need a **public Workflow API** for login and registration fields because the user is not authenticated at that point.

Therefore, a login access token cannot be sent for these APIs.

APIs

GET `/api/v1/workflows/public?type=auth_login`

GET `/api/v1/workflows/public?type=auth_register`

Expected Behaviour

Item	How it should work
Authentication	No user token required
<code>type</code>	<code>auth_login</code> or <code>auth_register</code>
<code>tenant_id</code>	Optional, only if login/signup fields differ by organisation
Result	Latest published login or registration workflow
No workflow found	Mobile shows the existing default email/password or signup fields

For example:

GET `/api/v1/workflows/public?type=auth_login`

should return the latest published login workflow without requiring the user to be authenticated.

6. Workflow Is Not Limited to Forms

The Workflow API should **not be considered a form-only functionality**.

A workflow may contain:

- Forms
- Fields
- UI components
- Steps
- Conditions
- Rules
- Actions
- Instructions
- Navigation logic
- Conditional behaviour
- Other workflow configurations

The purpose of the Workflow API is to allow the mobile application to dynamically understand **what needs to be shown and what the user needs to do next**, rather than limiting it to dynamically generated forms.

Example — Product Purchase Flow

For example, a product purchase workflow could define a sequence such as:

```
Select Product
      ↓
Choose Variant
      ↓
Enter Details
      ↓
Check Conditions
      ↓
Select Delivery Option
      ↓
Review Order
      ↓
Confirm Purchase
```

Based on the user's selection, the workflow could dynamically determine the next step.

For example:

```
User selects "Home Delivery"
      ↓
```

Show Delivery Address



Ask for Address Details



Show Delivery Slot

But if the user selects:

User selects "Store Pickup"



Skip Delivery Address



Show Pickup Location



Show Pickup Date

In this case, the workflow is not simply returning fields. It is defining the **logic and sequence of the process**.

Similarly, a workflow could support conditions such as:

- If the user selects a particular option, show a specific next step.
- Based on user input, dynamically skip a step.
- Add an additional step based on a condition.
- Change the UI or fields based on previous answers.
- Display different instructions based on user selections.
- Require a particular action before allowing the user to continue.
- Determine the next step dynamically.
- Guide the user through a sequence of actions required to complete a task.
- Support future workflow logic without requiring a new mobile release for every change.

Therefore, the Workflow API response should be designed as a **generic workflow definition**, not only as a collection of dynamic forms/fields.

7. Expected Workflow Response Design

The response should ideally be extensible enough to represent something like:

```
{  
  "workflow_id": "workflow-123",  
  "tenant_id": "tenant-456",  
  "type": "product",  
  "version": 5,  
}
```

```

"status": "published",
"steps": [
  {
    "id": "step-1",
    "type": "form",
    "title": "Product Details",
    "fields": []
  },
  {
    "id": "step-2",
    "type": "condition",
    "rules": []
  },
  {
    "id": "step-3",
    "type": "instruction",
    "content": "Review your order before continuing."
  },
  {
    "id": "step-4",
    "type": "action",
    "actions": []
  }
]
}

```

The exact response structure can be decided by the Workflow team.

The important requirement is that the structure should be **extensible** and should not assume that every workflow is only a form.